Proyecto Final de Sistemas embebidos

Sistema de Monitoreo Ambiental basado en ESP8266

Índice

Índice **2**
Resumen **3**
Introducción **3**
Objetivos **4**
Objetivo general 4
Objetivos específicos 4
Alcance y limitaciones **4**
Materiales y recursos utilizados **4**
Arquitectura y Diseño del Sistema **5**
A. Arquitectura General 5
Diagrama de bloques 5
Diagrama de las conexiones 5
Fotos de las conexiones 6
Diagrama de estados 7
Aplicación 7
B. Diseño de Hardware 8
Conexiones de los pines 9
C. Diseño de Firmware / Software 9
Pruebas y validación **10**
Problemas encontrados y soluciones aplicadas **10**
Resultados obtenidos **10**
Conclusiones **11**
Referencias y Anexos **11**
Código 11
Métodos de WiFi 11
Métodos de Temperature 14
Métodos del LED RGB 14
Métodos de Fan 15
Métodos de Main 15

Resumen

Este trabajo presenta el desarrollo de un sistema embebido basado en el microcontrolador ESP8266, orientado al monitoreo ambiental y control de actuadores con conectividad Wi-Fi para su gestión remota mediante una aplicación móvil. El prototipo integra sensor de temperatura y humedad, sensor de luz y actuadores como un LED RGB y la simulación visual de un sistema de ventilación.

Para optimizar el rendimiento, se implementó la manipulación de los registros y evitar el uso innecesario de recursos. Los registros que se utilizaron para la manipulación fueron el GPOS y GPOC, que son los registros con los que cuenta el microcontrolador. Los métodos con los que cuenta la librería de Arduino son pinMode() y digitalMode() que son ineficientes, entonces la mejor opción es modificar los registros.

En cuanto a la arquitectura de software, se diseñó un esquema Round Robin con quantum personalizado, asegurando ventanas de ejecución controladas para cada módulo (sensores, actuadores y comunicaciones) sin bloquear el procesador. Toda la solución se estructuró modularmente bajo el entorno PlatformIO, aplicando un patrón común de inicialización y ejecución secuencial. Asimismo, el documento aborda las decisiones de diseño ante limitaciones de hardware, tales como la mitigación de falsos positivos en sensado e interferencias electromagnéticas mediante la simulación lógica de componentes.

Introducción

El proyecto tiene como objetivo automatizar la temperatura y la luminosidad de una habitación por medio de una aplicación de celular. Cuenta con un sensor de temperatura y humedad, un sensor de luminosidad, como datos de entrada. Como datos de salida, un LED RGB, un LED para indicar que el sistema está funcionando y un ventilador, el cual es simulado. Todos estos dispositivos están conectados al microcontrolador ESP8266, el cual cuenta con WiFi y esto permite que podamos obtener datos y modificarlos por medio de una aplicación por medio de WiFi. La idea del proyecto es que siempre esté conectado a la corriente, ya que la placa funciona con 3V.

El proyecto surgió con componentes que tenía en mi casa. Me propuse utilizar lo que tenía al alcance, entonces surgió esté proyecto. Con respecto al software, como necesitaba una aplicación muy simple pero que permite conexión por medio de WiFi, la aplicación desde la página Mit Edu, cumplía con mis expectativas.

El sistema se orienta a mostrar una aplicación práctica de programación embebida, control digital de salidas y organización de tareas en un sistema con ESP8266. Asimismo, se tuvo en cuenta la restricción de pines del microcontrolador y la necesidad de evitar conflictos con otros componentes del sistema.

Objetivos

Objetivo general

Es diseñar e implementar un sistema de control de ventilación basado en ESP8266 que permita activar o desactivar su funcionamiento mediante un switch, lo cual permite encender el ventilador según la temperatura de la habitación que supere el umbral establecido por medio de la aplicación. Además, cuenta con un LED para obtener el estado operativo del dispositivo.

Objetivos específicos

- La implementación de un mecanismo de activación y desactivación general del sistema mediante el switch.
- Lectura de la temperatura ambiente de la habitación utilizando un sensor compatible con la plataforma.
- Encendido del ventilador cuando la temperatura sea mayor al umbral configurado.
- Utilización de un LED para identificar si el sistema se encuentra encendido y el ventilador funcionando.
- Organización de la ejecución de tareas mediante una planificación adecuada.
- Evitar conflictos de pines con otros módulos del proyecto.

Alcance y limitaciones

El sistema resuelve el control básico de ventilación e iluminación en función a umbrales configurados por medio de la aplicación de celular. El alcance completa la lectura de sensores, la activación de una salida digital para el ventilador y la señalización visual mediante LED.

No se incluyeron, como parte del desarrollo, funciones de monitoreo remoto, almacenamiento de datos, interfaz gráfica integrada en la placa, ni regulación de velocidad del ventilador. El diseño se centró en una solución funcional, robusta y de implementación sencilla pero con la posibilidad de agregar la regulación de velocidad de ventilador y cambiar o agregar algún sensor más.

Materiales y recursos utilizados

Los componentes que fueron utilizados para poder llevar a cabo el sistema son:

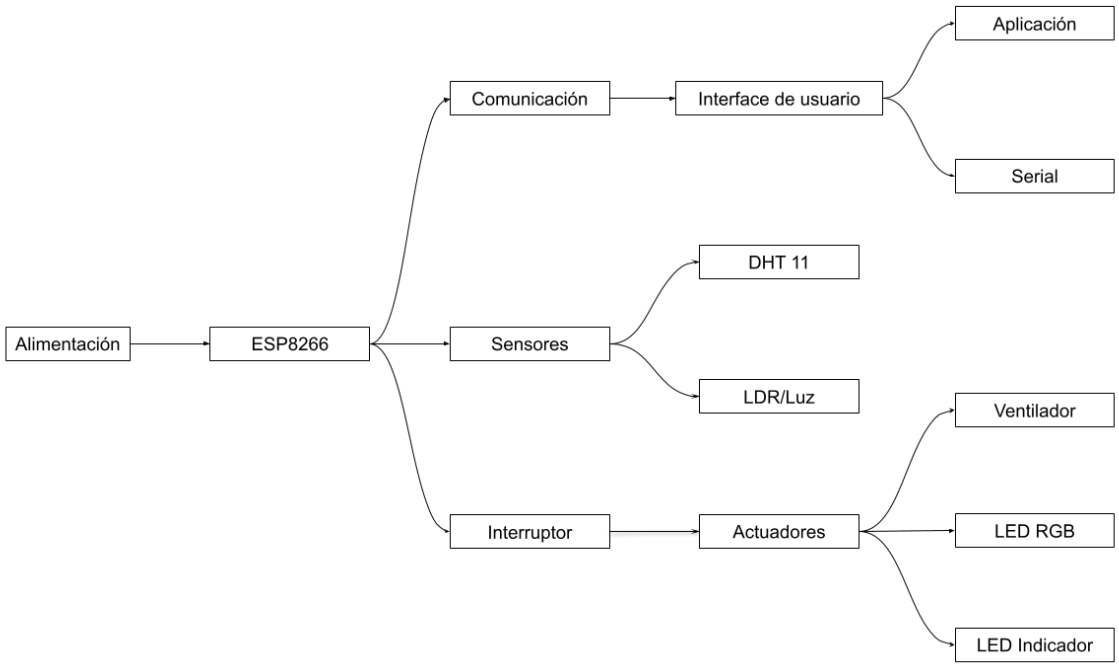
- Microcontrolador ESP8266.
- Sensor de temperatura y humedad, DHT11.
- Switch de cuatro pines.
- LED azul.
- Ventilador de cuatro pines

- Transistor MOSFET.
- Dos resistencias 500 ohms.
- Una resistencia de 10k ohms.
- LED RGB.

Arquitectura y Diseño del Sistema

A. Arquitectura General

Diagrama de bloques



El diagrama de bloques del proyecto.

Diagrama de las conexiones

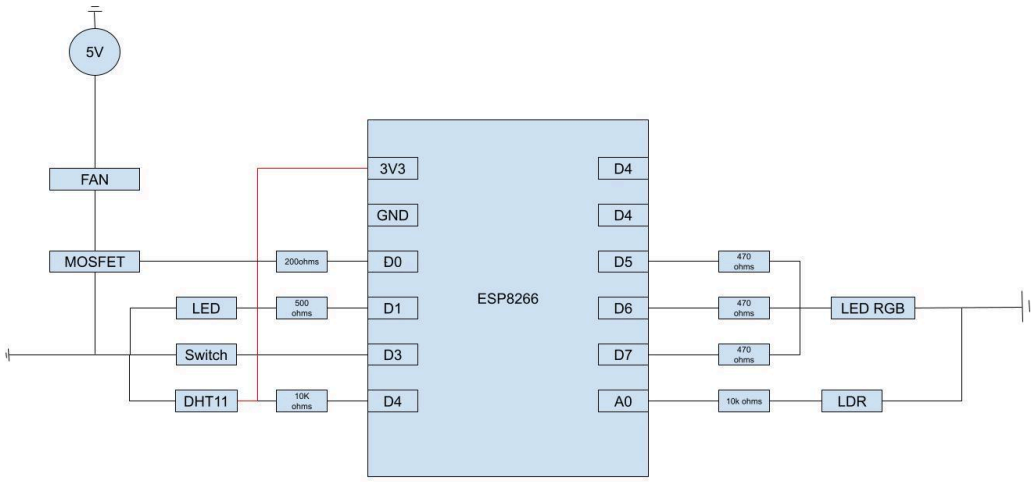
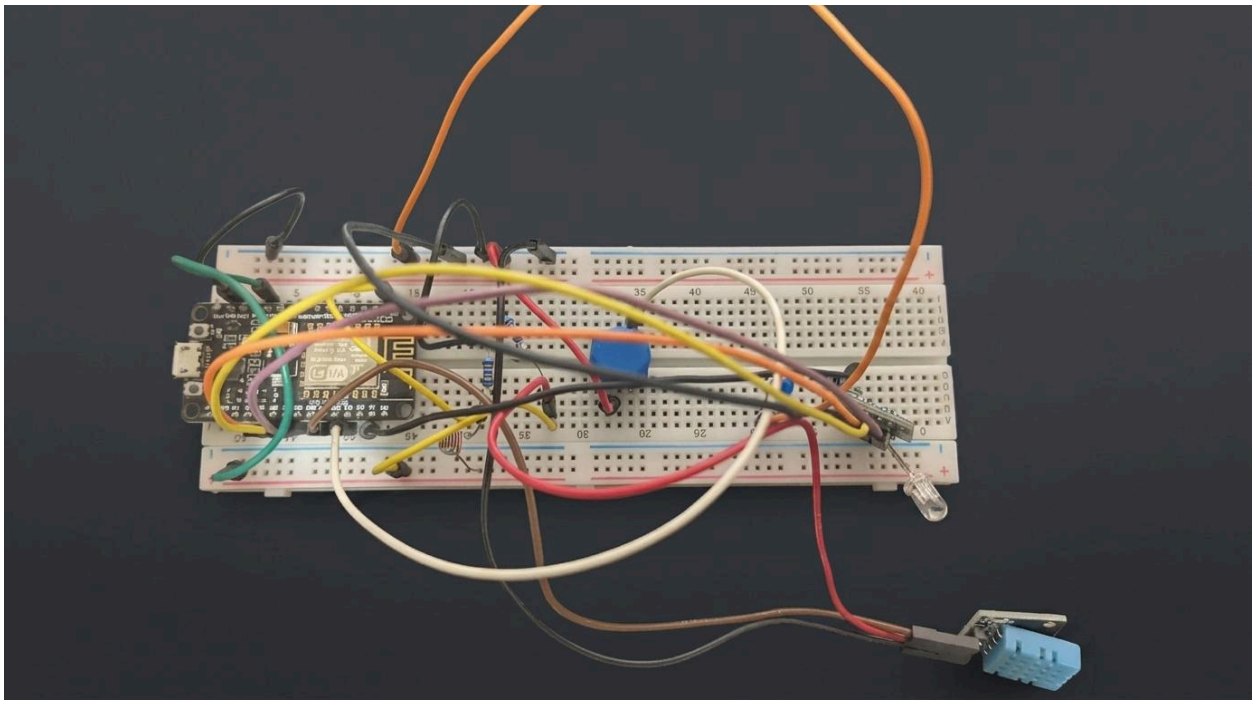


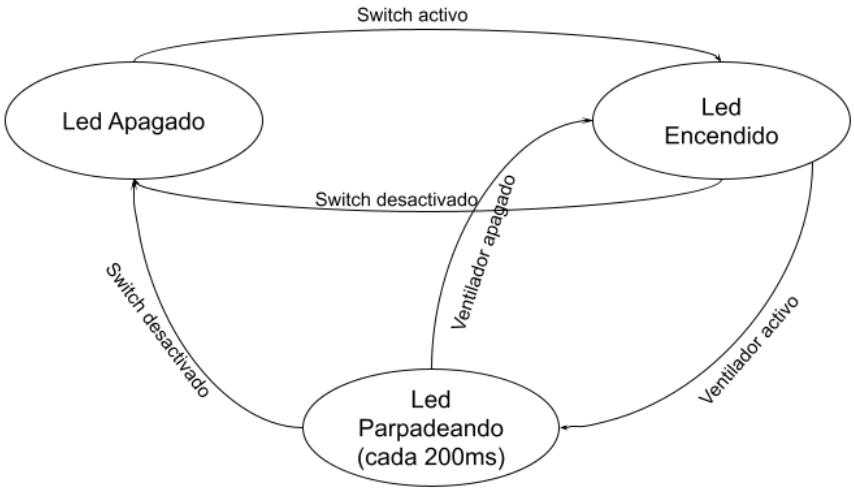
Diagrama de la conexión de forma simplificada

Fotos de las conexiones



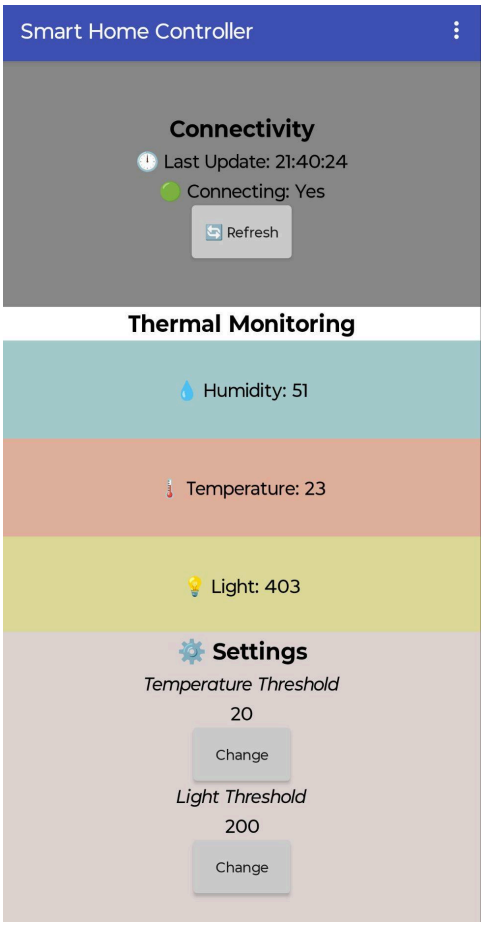
Las conexiones que realice, sin contar con ventilador ya que no contaba con uno.

Diagrama de estados



Es el diagrama de estados del LED, el cual indica si el sistema está encendido y si se encuentra el ventilador encendido.

Aplicación



La aplicación muestra la última actualización con el dispositivo. El botón *refresh* permite actualizar rápidamente los datos.

En el apartado *Thermal Monitoring*, solo muestra la humedad, temperatura y la luz de la habitación. Es decir, solo lee los datos de los sensores.

En el apartado de *Settings*, permite establecer los umbrales para la luz y la temperatura.

La aplicación es independiente de si está encendido el switch o no. Siempre se pueden consultar los datos, mientras ESP8266 se encuentre conectada.

B. Diseño de Hardware

Cuando empecé a investigar sobre la placa, noté que tiene un mejor rendimiento que un ATmega 328P pero es inferior a la placa que habíamos utilizado en la cátedra. Cuenta con una memoria RAM y flash suficiente para el proyecto, además de contar con la comunicación mediante protocolos webs como HTTP, MQTT, TCP/IP, Websockets. Admite varios dispositivos de entrada y salida, lo que permite que sea modular a las funciones que queramos agregar al proyecto. Cuenta con modos como Deep Sleep pero en el caso de esté proyecto, no es necesario, ya que lo pensé con la idea de utilizarlo en una habitación con corriente.

Como desventajas, no encontré varias como para cuestionarme cambiar de placa pero cuenta con una importante qué es que las GPIO funcionan a 3,3V y no son tolerantes a 5V. Esto hay que tenerlo en cuenta, ya que si se conecta a 5V se puede dañar el proyecto.

El corazón del proyecto es, como mencioné anteriormente, el microcontrolador ESP8266, el cual cuenta con una placa WiFi integrada, lo que permite realizar conexiones por medio de una nube. Luego, con el sensor de luminosidad y de temperatura/humedad, que son dispositivos que obtienen información.

El sensor de luminosidad es un sensor de fotorresistencia (LDR-Light Dependent Resistor) que varía según la luz que recibe. El funcionamiento consiste en que en la oscuridad tiene mucha resistencia, y bajo la luz su resistencia baja drásticamente. Para el funcionamiento adecuado se necesita conectarla a una resistencia de 10k Ohms para convertir ese cambio de resistencia en una variación de voltaje que el pin analógico pueda leer.

El sensor de temperatura y humedad es el DH11 montado en un módulo (PCB). Su función principal es medir la humedad relativa, es decir, el vapor de agua en el aire y la temperatura. Para esto, cuenta con un componente llamado termistor para medir el calor del ambiente. Los rangos en los que se mide la humedad son 20% a 80% con un margen de error del 5%. Para la temperatura mide de 0°C a 50°C, con un margen de error del 2°C. Además funciona entre 3.3V y 5V.
Para un uso hogareño y de interior, cumple porque aunque no mida bajo cero o humedad alta/baja, son temperaturas que suelen haber en un hogar tradicional.

El sensor de movimiento es un sensor infrarrojo pasivo (PIR - HC-SR501). Su función es detectar el movimiento de personas o animales midiendo los cambios en los niveles de radiación infrarroja, es decir, el calor corporal. Posee una cúpula blanca que es una lente de Fresnel, que divide el área en múltiples zonas para que el sensor pueda observar cuando algo caliente cruza de una zona a otra.
El dispositivo cuenta con distintos ajustes, potenciometros amarillo que uno ajusta la sensibilidad, es decir, la distancia de detección, la cual es de hasta 7 metros. Otro es el tiempo que la señal permanece activa tras detectar movimiento. Además un jumper amarillo que permite elegir entre disparo repetitivo o único.
Esté sensor es muy sensible, entonces tuve un problema con ESP8266 que me generaba ruido, es decir, me daba falsos positivos por más que lo separe la mayor distancia posible con

la configuración menos sensible al movimiento. Para solucionar esté inconveniente, decidí agregar el switch de cuatro pines para indicar que quiero que el sistema encienda la luz o el ventilador cuando se activa o enciende la luz. Es una forma de indicar que hay alguien en esa habitación, ya que sin esté mecanismo se malgasta energía.

También se encuentra el componente denominado LED RGB, que simula el LED de una lámpara o LED de una casa. Contiene tres diodos, uno de color Rojo, Verde y Azul. Esto permite mezclar los diferentes colores e intensidades para poder generar nuevos colores.

Conexiones de los pines

- D0: Salida de control del ventilador
- D1: LED indicador del sistema (Luz constante si se encuentra encendido el sistema - luz parpadeando si el ventilador se encuentra encendido)
- D3: Entrada del switch.
- D4: Lectura del sensor de temperatura y humedad DHT11
- D5:
- D6:
- D7:
- GND: Masa común para todo el circuito.
- 3V3: Voltaje al que funcionan la mayoría de los componentes.

Las conexiones se pueden ver de forma gráfica en el diagrama.
Un detalle importante a tener en cuenta es que el ventilador no puede conectarse de forma directa al GPIO de ESP8266, ya que la corriente que requiere, excede la capacidad de entrega de la placa. Por dicho motivo, se utiliza una etapa intermedia de potencia.

C. Diseño de Firmware / Software

En cuestión de software, decidí implementarlo con Round Robin ya que es la arquitectura que se suele utilizar en proyectos de Arduino. La única modificación que hice fue modificar el quantum, ya que cuando iba realizando las pruebas y agregando funciones, me encontré que se iba demorando, entonces para poder evitar está situación, decidí personalizar el quantum e ir probando cual es el más apropiado para las funciones con las que cuenta. Cada tarea tiene su propio quantum para poder optimizar el uso del hardware. Entonces esté tipo de arquitectura, se adapta muy bien al sistema, ya que por más que se vuelve lento cuando se van agregando sensores, se puede personalizar con el quantum.
Las demás arquitecturas fueron analizadas pero tomé la decisión de que eran muy complejas para un sistema simple y que no necesita responder de forma milimétrica como para agregar un sistema de tiempo real.

El entorno de desarrollo fue Visual Studio Code con la extensión de PlatformIO, la cual ya fue con la librería para el microcontrolador ESP8266, además de conocer el software me permite mejor manejo de las herramientas.
La implementación se realizó en lenguaje C++ sobre el framework Arduino. El programa organiza sus acciones mediante tareas periódicas, entre las que se destacan la lectura de temperatura, el control del ventilador, la lectura del switch y la actualización del LED indicador. Está estructura permite mantener una ejecución ordenada sin bloquear de forma innecesaria el ciclo principal.

Para la aplicación de celular, encontré una página web denominada APP Inventor, en la cual podes realizar aplicaciones para celulares con Android de forma muy simple y cuenta con las herramientas que necesitaba. La programación es por bloques y con una interfaz bastante intuitiva que resolvía las necesidades para la aplicación.

Pruebas y validación

Se realizaron pruebas funcionales sobre cada una de las partes principales del sistema. En primer lugar, se verificó el correcto funcionamiento del switch. Luego se comprobó la respuesta del LED en sus tres estados definidos. Finalmente se validó el comportamiento del ventilador en función de la temperatura y el estado activo del sistema.

Las pruebas permitieron confirmar que el ventilador se activa únicamente cuando la temperatura supera el umbral y el sistema está habilitado. Asimismo, se confirmó que el LED parpadea de manera visible cuando el ventilador entra en funcionamiento.

Problemas encontrados y soluciones aplicadas

Además del problema mencionado sobre el sensor de movimiento, también me encontré con el conflicto de pines, debido a que D2 estaba comprometido por otro módulo del proyecto. Por esté motivo, el LED fue asignado a D1, cuando normalmente se le suele asignar el pin D2.
Otro punto importante fue la actualización del estado visual del LED cuando el ventilador se encontraba activo. Para resolverlo, se aseguró que la rutina de actualizada del LED se ejecutará de forma periódica mientras el sistema permaneciera activo. De esté modo, el parpadeo se mantiene durante todo el tiempo de funcionamiento del ventilador.

Resultados obtenidos

El sistema final responde correctamente a la idea establecida principalmente. El switch habilita o deshabilita el funcionamiento general. El ventilador se activa cuando se supera el umbral y el LED indica visualmente el estado del sistema. La solución obtenida es estable, comprensible y coherente con los objetivos del proyecto.

Conclusiones

El proyecto permitió desarrollar una solución de automatización simple pero efectiva, capaz de responder a condiciones térmicas y de informar el estado mediante una señal visual.
El trabajo me aportó experiencia práctica en programación como también en problemas de hardware. Como mejora futura, sería posible incorporar monitoreo remoto, almacenamiento de datos o una interfaz de usuario más completa.

Referencias y Anexos

- https://docs.platformio.org/en/latest/platforms/espressif8266.html
- https://es.wikipedia.org/wiki/ESP8266
- https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf
- https://kmindustrialcorp.com/blog/ldr-sensor-overview/
- https://projecthub.arduino.cc/semsemharaz/interfacing-rgb-led-with-arduino-b59902
- https://appinventor.mit.edu/

Código

El link del repositorio: https://github.com/pintosmicaela/embedded-systems/

Métodos de WiFi

En wifi.h cuentan con los siguientes métodos:

Descripción: Inicializa el puerto serie, conecta el ESP8266 a la red Wifi, sincroniza la hora mediante NTP e inicia el servidor web.

Variables de entrada: No tiene.

Salida: No tiene.

```
void setup_wifi();
```

Descripción: Atiende las solicitudes HTTP entrantes.

Variables de entrada: No tiene.

Salida: No tiene.

```
void loop_wifi_server();
```

Descripción: Muestra por el puerto serie el estado actual de la conexión WiFi.

Variables de entrada: No tiene.

Salida: No tiene.

```
void status_wifi();
```

Descripción: Solicita la hora a un servidor NTP y espera hasta que sea válida o se agoten los intentos.

Variables de entrada: No tiene.

Salida: No tiene.

```
void sync_hour();
```

Descripción: Determina si la hora fue sincronizada correctamente.

Variables de entrada: No tiene.

Salida: Bool.

```
bool time_is_synced();
```

Descripción: Obtiene la hora actual en formato Unix/Epoch.

Variables de entrada: No tiene.

Salida: time_t

```
time_t get_epoch_time();
```

Descripción: Obtiene la fecha y hora actual como texto con el formato AAAA-MM-DD HH:MM:SS.

Variables de entrada: No tiene.

Salida: String

```
String get_time_string();
```

Descripción: Obtiene la temperatura actual en grados Celsius.

Variables de entrada: No tiene.

Salida: float

```
float get_current_temperature();
```

Descripción: Obtiene la humedad actual expresada en porcentajes.

Variables de entrada: No tiene.

Salida: float

```
float get_current_humidity();
```

Descripción: Obtiene el valor actual de luminosidad leído por el ADC.

Variables de entrada: No tiene.

Salida: int

```
int get_current_light();
```

Descripción: Obtiene el umbral de temperatura configurado para activar el ventilador.

Variables de entrada: No tiene.

Salida: float

```
float get_temperature_threshold();
```

Descripción: Obtiene el umbral de luminosidad configurado.

Variables de entrada: No tiene.

Salida: int

```
int get_light_threshold();
```

Descripción: Modifica el umbral de temperatura.

Variables de entrada: float value (nuevo valor de umbral)

Salida: No tiene.

```
void set_temperature_threshold(float value);
```

Descripción: Modifica el umbral de la luminosidad.

Variables de entrada: int value (nuevo valor de umbral)

Salida: No tiene.

```
void set_temperature_threshold(float value);
```

Métodos de Temperature

Los métodos que tiene temperature.h son:

Contiene una estructura DHT_Data, que la componen uint8_t humidity, uint8_t temperature y bool success.

Descripción: Configura el GPIO 2 (D4) para comunicarse con el sensor DHT , activa el pull-up y prepara el pin como salida.
Variables de entrada: No tiene.
Salida: No tiene.

```
void setup_dht();
```

Descripción: Verifica los valores obtenidos por el sensor con los limites establecidos.
Variables de entrada: t_limit: limite de la temperatura; h_limit: limite de la humedad.
Salida: bool.

```
bool check_tresholds (float t_limit, float h_limit);
```

Descripción: Obtiene la temperatura y la humedad desde el sensor. Mide los pulsos y reconstruye los 40bits enviados y verifica el checksum.
Variables de entrada: No tiene.
Salida: DHT_Data

```
DHT_Data read_dht_raw();
```

Métodos del LED RGB

Los métodos que tiene led_rgb.h son:

Descripción: Configura los pines del LED RGB como salida y lo apaga al iniciar.
Variables de entrada: No tiene.
Salida: No tiene.

```
void setup_led();
```

Descripción: Apaga los tres colores del LED RGB.
Variables de entrada: No tiene.
Salida: No tiene.

```
void turn_off();
```

Descripción: Enciende o apaga cada canal según su valor.

Variables de entrada: int r: valor del canal rojo; int g: valor del canal verde; int b: valor del canal azul.
Salida: No tiene.

```
void set_color(int r, int g, int b);
```

Métodos de Fan

Los métodos que tiene fan.h son:

Descripción: Configura los pines del ventilador y el LED indicador como salidas, y comienza con ambos apagados.
Variables de entrada: No tiene.
Salida: No tiene.

```
void setup_fan();
```

Descripción: Actualiza el estado del ventilador y modifica físicamente la salida correspondiente.
Variables de entrada: bool active: indica si se debe activar o apagar.
Salida: No tiene.

```
void set_fan_active(bool active);
```

Descripción: Consulta el estado actual guardado del ventilador.
Variables de entrada: No tiene.
Salida: bool

```
bool is_fan_active();
```

Descripción: Actualiza el LED indicador según el estado del sistema del ventilador.
Variables de entrada: bool system_active: indica si se encuentra activo el sistema.
Salida: No tiene.

```
void update_fan_indicator (bool system_active);
```

Métodos de Main

Los método que se encuentran en main.cpp son:

Descripción: Enciende el LED RGB en blanco.
Variables de entrada: No tiene.

Salida: No tiene.

```
void light_on();
```

Descripción: Apaga el LED RGB.
Variables de entrada: No tiene.
Salida: No tiene.

```
void light_off();
```

Descripción: Determina si la tarea debe ejecutarse.
Variables de entrada: const Task&: tarea que se desea verificar; unit8_t now_ms: tiempo en millis.
Salida: bool

```
bool task_due (const Task&, unit8_t now_ms);
```

Descripción: Ejecuta una tarea, actualiza el indicador del ventilador y pasa a la siguiente tarea.
Variables de entrada: No tiene.
Salida: No tiene.

```
void run_round_robin();
```

Descripción: Lee el sensor DHT. Si es un valor válido, actualiza la temperatura.
Variables de entrada: No tiene.
Salida: No tiene.

```
void task_read_temperature();
```

Descripción: Lee el sensor DHT. Si es un valor válido, actualiza la humedad.
Variables de entrada: No tiene.
Salida: No tiene.

```
void task_read_humidity();
```

Descripción: Lee el valor del sensor de luz mediante el ADC.
Variables de entrada: No tiene.
Salida: No tiene.

```
void task_read_light();
```

Descripción: Comprueba si la temperatura supera el umbral establecido. En caso afirmativo, se enciende el ventilador.
Variables de entrada: No tiene.
Salida: No tiene.

```
void task_check_temperature_alert();
```

Descripción: Controla la luz según el estado del sistema y el nivel de luminosidad medido.
Variables de entrada: No tiene.
Salida: No tiene.

```
void task_control_light();
```

Descripción: Lee el interruptor, detecta los cambios de estado y activa o desactiva el sistema.
Variables de entrada: No tiene.
Salida: No tiene.

```
void task_read_switch();
```

Descripción: Obtiene la temperatura actual.
Variables de entrada: No tiene.
Salida: float

```
float get_current_temperature();
```

Descripción: Obtiene la humedad actual.
Variables de entrada: No tiene.
Salida: float

```
float get_current_humidity();
```

Descripción: Obtiene el valor actual de luminosidad.
Variables de entrada: No tiene.
Salida: int

```
int get_current_light();
```

Descripción: Obtiene el umbral de la temperatura.
Variables de entrada: No tiene.
Salida: float

```
float get_temperature_threshold();
```

Descripción: Obtiene el umbral de la luz.
Variables de entrada: No tiene.
Salida: int

```
int get_light_threshold();
```

Descripción: Actualiza el umbral de la temperatura.
Variables de entrada: float value: nuevo umbral.
Salida: No tiene.

```
void set_temperature_threshold (float value);
```

Descripción: Actualiza el umbral de la luminosidad.
Variables de entrada: int value: nuevo umbral.
Salida: No tiene.

```
void set_light_threshold (int value);
```

Descripción: Inicializa el puerto serie, los sensores, ventilador, WiFi, LED RGB y el switch.
Variables de entrada: No tiene.
Salida: No tiene.

```
void setup();
```

Descripción: Ejecuta el planificador de tareas.
Variables de entrada: No tiene.
Salida: No tiene.

```
void loop();
```